



JAVA



Reusability is the power of OOP!

1

INHERITANCE



NOTES



What is Inheritance?

Inheritance is an Object-Oriented Programming (OOP) concept where one class acquires the **properties and behaviors** of another class.

Simple Definition

Inheritance allows a child class to reuse the variables and methods of a parent class.



Why do we use Inheritance?

- ✓ **Code Reusability** - Write once, use many times.
- ✓ **Maintainability** - Changes in parent reflect in all children.
- ✓ **Extensibility** - Add new features in child class.
- ✓ **Polymorphism** - Enables method overriding.

Child extends Parent



② SYNTAX



2

```
class Parent {  
    // properties and methods  
}  
  
class Child extends Parent {  
    // additional properties and methods  
}
```

extends

The 'extends' keyword is used to inherit one class from another.

③ EXAMPLE



```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog dog = new Dog();  
  
        dog.eat();    // inherited from Animal  
        dog.bark();  // Dog's own method  
    }  
}
```

OUTPUT

Animal is
eating
Dog is
barking



Real-world Analogy

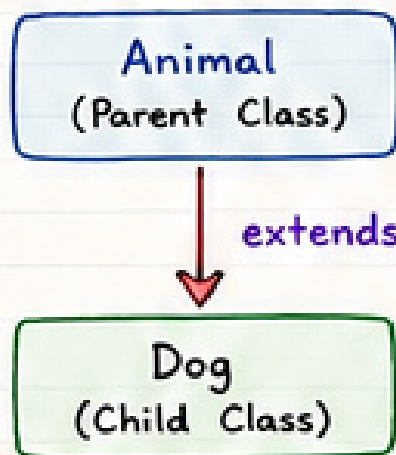
- Dog IS-A Animal
- Car IS-A Vehicle
- Developer IS-A Employee
- SavingsAccount IS-A BankAccount



④ HOW IT WORKS ?



③

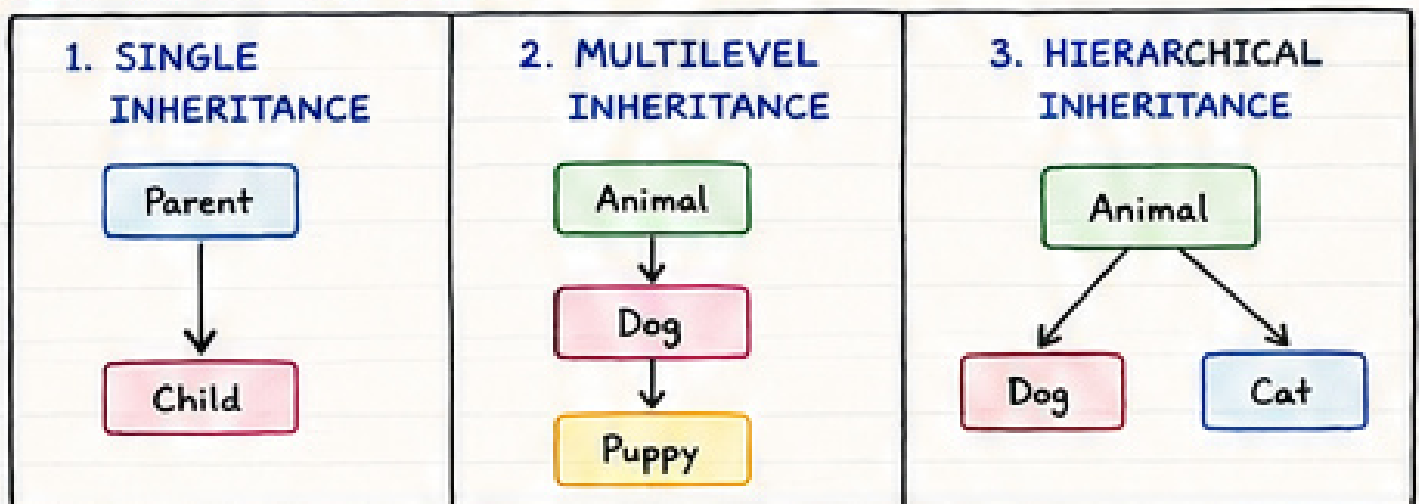


Explanation

- Animal → Parent / Superclass
- Dog → Child / Subclass
- Dog inherits the eat() method from Animal.
- Dog can also define its own methods like bark().



⑤ TYPES OF INHERITANCE IN JAVA



Java does NOT support multiple inheritance using classes

```
    dog.bark(); // Dog's own method  
  }  
}
```



⑥ SUPER KEYWORD

4

- The `super` keyword is used to refer immediate parent class instance variables and methods.
- It is also used to call parent class constructor.



Example:

```
class Animal {  
    String type = "Animal";  
    Animal() {  
        System.out.println("Animal Constructor");  
    }  
}  
  
class Dog extends Animal {  
    String type = "Dog";  
    Dog() {  
        super(); // calls parent constructor  
        System.out.println("Dog Constructor");  
    }  
    void show() {  
        System.out.println("Parent type: " + super.type);  
        System.out.println("Child type: " + this.type);  
    }  
}
```

OUTPUT

```
Animal  
Constructor  
Dog  
Constructor  
Parent type:  
Animal  
Child type:  
Dog
```



super refers
to parent

Java does NOT support multiple inheritance using classes.

✗ Not allowed

```
class C extends A, B {  
}
```

↩ Multiple inheritance-like behavior
can be achieved using interfaces.

⑦ IMPORTANT POINTS



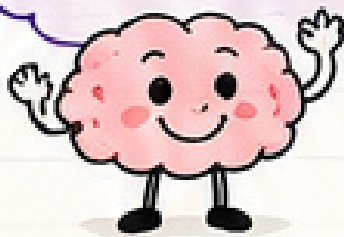
⑤



- Use **extends** for class inheritance.
- A child class can access eligible members of its parent.
- private members cannot be accessed directly by the child class.
- Constructors are **NOT** inherited.
- Java classes support **SINGLE** inheritance.
- Java supports **Multilevel** and **Hierarchical** inheritance.
- Every Java class ultimately inherits from the **Object** class.
- Inheritance represents an **IS-A** relationship.



REMEMBER



Parent is also called **Superclass**
or **Base** class.

Child is also called **Subclass**
or **Derived** class.



Practice more, **Code** more, **Become**

⑧ METHOD OVERRIDING (WITH INHERITANCE)

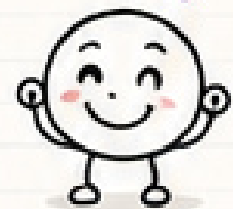
6

- When child class provides its own implementation of parent class method, it is called Method Overriding.
- It is used to achieve Runtime Polymorphism.

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
Animal a = new Dog();    // reference of parent, object of child  
a.sound();               // Output: Dog barks
```

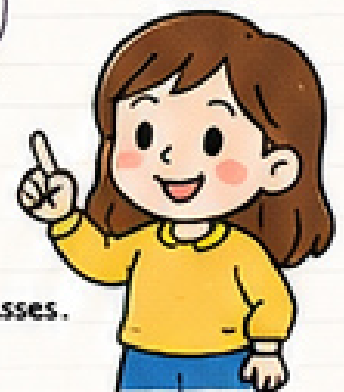
@Override
annotation is
optional but
recommended.



⑨ INTERVIEW POINT

Q. What is inheritance in Java?

Ans. Inheritance is an OOP mechanism in which one class acquires the properties and behaviors of another class using the **extends** keyword. It promotes code reusability and establishes an IS-A relationship between parent and child classes.



a Better Java Developer! ❤️

